

राष्ट्रीय प्रौद्योगिकी संस्थान दिल्ली
NATIONAL INSTITUTE OF TECHNOLOGY ,
DELHI

PROJECT REPORT

on

**TOPIC – RFID ENABLED EV PARKING AND
CHARGING SYSTEM**

Submitted

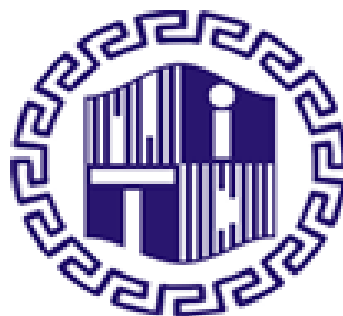
By

Priya Sadulwar (Roll no. - 241230045)

Sharvari Kulkarni (Roll no. - 241230035)

Supervised by –

Dr. Anmol Ratna Saxena



PROJECT APPROVAL FOR B.TECH

This Project work entitled “RFID enabled EV parking and charging system” by Priya Sadulwar , Sharvari Kulkarni submitted in the partial fulfilment of the requirements of the course EEPB 255.

Supervisor

DR. ANMOL RATNA SAXENA

(Associate Professor)

Date : 12th May 2026

Place : National Institute of Technology , Delhi

DECLARATION

We, Priya Sadulwar (241230045) and Sharvari Kulkarni (241230049), students of B.Tech in Electrical Engineering (Semester 4) at the National Institute of Technology, Delhi, hereby solemnly declare that the project report entitled:

“RFID Enabled smart parking and charging station”

is an original piece of work carried out by us during the academic year 2024-25, under the supervision of Associate Professor Dr. Anmol Ratna Saxena, Department of Electrical Engineering, National Institute of Technology, Delhi.

We further declare that:

1. This work has not been submitted, either in part or full, to any other university or institution for the award of any degree or diploma.
2. All sources of information and literature used in this work have been duly cited and referenced in accordance with academic standards.
3. The simulation work ,hardware and web dashboards in this report are genuine and carried out independently.
4. All data, results, and figures presented in this report are genuine and have not been fabricated or manipulated.
5. We take full responsibility for the content of this report and accept the consequences of any violation of the above declarations.

Place: New Delhi Date: 12th May, 2026

TABLE OF CONTENTS

DECLARATION.....	3
TABLE OF FIGURES	5
ACKNOWLEDGEMENT	6
ABSTRACT.....	7
CHAPTER 1 : INTRODUCTION	8
1.1 Background and Motivation	8
1.2 Problem Statement	8
1.3 Objectives of the Project.....	8
1.4 Scope of the Project	9
CHAPTER 2 : LITERATURE REVIEW	10
2.1 Literature Review.....	10
2.2 Existing System	11
2.3 Proposed System.....	11
CHAPTER 3 : METHODOLOGY	12
3.1 Overall System Approach	12
3.2 Block Diagram	12
3.3 Data Flow.....	13
3.4 System Working — Step by Step.....	13
3.4.1 RFID Authentication.....	13
3.4.2 Slot Allocation Logic	13
3.4.3 Gate Automation	14
3.4.4 Slot Occupancy Detection.....	14
3.4.5 Charging Simulation	14
3.4.6 Firebase Data Upload.....	15
CHAPTER 4 : SIMULATION.....	16
CHAPTER 5 : HARDWARE MODEL	17
5.1 Components Used	17
5.2 Hardware Setup.....	18
CHAPTER 6 : SOFTWARE AND CLOUD INTEGRATION	19
6.1 Arduino IDE and Libraries.....	19
6.2 Firebase Realtime Database	19

6.3 Dashboard Design.....	20
6.3.1 Operator Dashboard.....	20
6.3.2 User Dashboard.....	21
CHAPTER 7 : RESULTS	22
7.1 Test Results	22
7.2 Observations	23
CHAPTER 8: CONCLUSION AND FUTURE SCOPE	24
8.1 Conclusion	24
8.2 Future Scope	24
REFERENCES	25

TABLE OF FIGURES

Figure 1: Block diagram	12
Figure 2 : Data flow	13
Figure 3 : Simulation	16
Figure 4 : Harware setup.....	18
Figure 5 : Firebase interface	20
Figure 6 : Operator dashboard	21
Figure 7 : User dashboard.....	21
Figure 8 : EV user portal while slots unavailability.....	23

LIST OF TABLES

Table 1 Firebase data	15
Table 2 Components	17
Table 3 : Libraries used.....	19
Table 4 : RFID authentication test	22
Table 5 : Slot allocation test.....	22
Table 6 : Charging simulation test	22

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who supported and guided us throughout the completion of this project.

First and foremost, we extend our deepest thanks to our project supervisor, Dr. Anmol Saxena, Assistant Professor, Department of Electrical Engineering, National Institute of Technology Delhi, for his invaluable guidance, continuous encouragement, and constructive feedback throughout the project. His technical expertise and insightful suggestions have been instrumental in shaping this project.

We are grateful to the Head of the Department of Electrical Engineering, NIT Delhi, for providing the necessary facilities and a conducive academic environment that enabled us to carry out this work effectively.

We also thank the laboratory staff of the Department of Electrical Engineering for providing access to the hardware laboratory and assisting us in setting up the experimental prototype. We are thankful to our classmates and peers for their support, discussions, and motivation during the course of this project. Finally, we are profoundly grateful to our families for their unwavering support, patience, and encouragement throughout our academic journey.

Priya Sadulwar Roll No. 241230045

Sharvari Kulkarni Roll No. 241230049

Department of Electrical Engineering National Institute of Technology Delhi Academic
Session: 2025–26

ABSTRACT

The increasing adoption of electric vehicles (EVs) across the world has created a strong demand for smart, automated, and efficient EV charging infrastructure. Traditional EV charging stations are mostly manual, unmonitored, and lack real-time data, which makes them difficult to manage in a smart city environment. This project presents the design and implementation of a Smart EV Charging Station Management System using the ESP32 microcontroller, RFID authentication, IR sensors, relay module, servo motor, and Firebase Realtime Database.

The system works as follows: When a user arrives at the charging station, they scan their RFID card. The ESP32 reads the card and checks slot availability. The system automatically assigns the nearest available slot and opens the entry gate using a servo motor. IR sensors continuously monitor all three charging slots. Once the vehicle occupies the assigned slot, the relay module activates and charging begins. Live data — including slot status, battery percentage, charging temperature, and estimated time remaining — is continuously uploaded to Firebase Realtime Database.

Two separate web dashboards have been developed using HTML, CSS, and JavaScript. The Operator Dashboard displays the complete status of all slots and all charging parameters, allowing station operators to monitor the entire station remotely. The User Dashboard shows only the relevant information for the authenticated user — assigned slot, charging status, battery level, temperature, and time remaining.

This project demonstrates a practical and low-cost IoT-based solution that can improve the efficiency of EV charging station management and can serve as a foundation for future smart city EV infrastructure.

CHAPTER 1 : INTRODUCTION

1.1 Background and Motivation

The world is shifting towards electric vehicles as a major solution to reduce air pollution and dependence on fossil fuels. Countries like India are investing heavily in EV infrastructure under programs like FAME (Faster Adoption and Manufacturing of Electric Vehicles). As the number of EVs on roads increases, the demand for efficient, smart, and automated charging stations is also growing rapidly.

However, most existing EV charging stations are basic and manual. There is no system to automatically allocate charging slots, no real-time monitoring of charging status, and no way for users or operators to remotely check the station conditions. This leads to problems like overcrowding, unmanaged parking, inefficient use of available charging slots, and poor user experience. The Internet of Things (IoT) offers a powerful solution to these problems. IoT-based systems can automate station operations, collect real-time data, and make it available through cloud platforms and web dashboards. This project is motivated by the need to design a simple, practical, and affordable IoT-based EV charging station management system.

1.2 Problem Statement

Current EV charging stations face several challenges:

- There is no automated method to assign charging slots to incoming vehicles.
- Users have no way to know in advance which slot is available or how long charging will take.
- Station operators cannot monitor the station remotely.
- Charging starts and stops manually, without any automation or verification.
- There is no real-time cloud monitoring of battery level, charging status, or temperature.

These problems make EV charging stations inefficient, especially during peak hours. There is a clear need for a smart, automated, and cloud-connected charging station management system

1.3 Objectives of the Project

The main objectives of this project are:

1. To implement RFID-based user authentication for secure entry into the EV charging station.
2. To automatically allocate available charging slots to authenticated users.
3. To automate the entry gate using a servo motor.
4. To detect slot occupancy using IR sensors.
5. To simulate EV charging using a relay module.
6. To monitor battery percentage, temperature, and estimated charging time in real time.

7. To upload all live data to Firebase Realtime Database.
8. To develop an Operator Dashboard for complete station monitoring.
9. To develop a User Dashboard for user-specific charging information

1.4 Scope of the Project

This project is a working prototype that demonstrates all the major features of a smart EV charging station. The scope includes:

- Embedded programming using ESP32 and Arduino IDE.
- Hardware integration of RFID, IR sensors, relay module, and servo motor.
- Cloud integration using Firebase Realtime Database.
- Web dashboard development using HTML, CSS, and JavaScript.
- Real-time data monitoring and visualization.

This project can be scaled up for real-world deployment with minor hardware and software upgrades.

CHAPTER 2 : LITERATURE REVIEW

2.1 Literature Review

Several research works have explored the use of IoT, RFID, and cloud platforms in smart parking and EV charging systems. The following section reviews some key ideas and existing works related to this project.

IoT-Based Smart Parking Systems: Early IoT-based parking systems used ultrasonic or IR sensors to detect vehicle presence in parking slots. These systems could show real-time slot availability on LED displays or mobile apps. The concept of using sensors for automatic slot detection forms the foundation of this project's slot monitoring module.

RFID in Access Control Systems: RFID technology has been widely used in access control and user authentication systems. In transportation, RFID cards are used in metro systems, toll plazas, and vehicle tracking. Using RFID for EV charging station authentication adds security and ensures that only authorized users can access charging services. The RC522 RFID module is a widely adopted, cost-effective choice for embedded systems.

Firebase in IoT Applications: Firebase Realtime Database is a cloud-hosted database by Google that supports real-time data synchronization. Many IoT projects use Firebase due to its simplicity, speed, and ability to sync data across web and mobile clients in milliseconds. ESP32 can communicate with Firebase over WiFi using REST API calls, making it a natural choice for this project.

EV Charging Management Systems: Several research papers have proposed smart EV charging systems with features like dynamic pricing, grid load balancing, and remote monitoring. While complex systems require industrial-grade hardware, simpler prototypes using microcontrollers like Arduino and ESP32 have been demonstrated as proof-of-concept models. This project builds on such lightweight prototype approaches while adding real-time cloud connectivity.

Web Dashboard for IoT Monitoring: Web-based dashboards built using HTML, CSS, and JavaScript with Firebase integration are commonly used for IoT monitoring. These dashboards can display live sensor data, charts, and control buttons. For this project, two separate dashboards — one for the operator and one for the user — have been developed to serve different purposes.

Research Gap: Most existing low-cost EV charging prototypes either focus only on hardware (sensor and relay control) or only on software (app-based booking). Very few prototypes integrate both real-time hardware control and live web-based dashboards with user-level and operator-level interfaces. This project addresses that gap by combining hardware automation with cloud monitoring and dual-dashboard access.

2.2 Existing System

Existing EV charging systems mainly provide basic charging facilities with limited automation and monitoring capabilities. In many charging stations, users must manually search for available parking or charging slots, which increases waiting time and reduces operational efficiency. Several modern charging networks use cloud-based monitoring systems, but most of these systems are expensive and designed for large-scale commercial deployment. Many low-cost prototype systems developed for educational or small-scale purposes focus only on either smart parking or charging management separately. As a result, features such as secure RFID authentication, automatic slot allocation, live charging monitoring, and real-time dashboard updates are often missing in a single integrated system. In addition, many traditional systems do not provide separate interfaces for users and operators, making monitoring and management less efficient.

2.3 Proposed System

The proposed system is an IoT-based Smart EV Charging Station Management System developed using ESP32, RFID technology, IR sensors, Firebase Realtime Database, relay module, servo motor, and web-based dashboards. The system is designed to automate charging station operations and provide real-time monitoring through cloud integration. When a user scans an RFID card, the ESP32 verifies the authentication and automatically assigns the nearest available charging slot. The servo motor then opens the entry gate, allowing the vehicle to enter the station.

IR sensors installed at each charging slot continuously monitor vehicle occupancy. Once the assigned slot becomes occupied, the relay module activates charging simulation. During charging, parameters such as battery percentage, charging temperature, and estimated charging time remaining are updated dynamically. All system data is transmitted to Firebase Realtime Database through WiFi communication provided by the ESP32.

The system includes two separate dashboards: an operator dashboard and a user dashboard. The operator dashboard displays the status of all charging slots along with charging information, while the user dashboard displays details related only to the assigned slot. Both dashboards receive live updates from Firebase without requiring page refresh. The proposed system provides a low-cost, automated, and efficient solution for smart EV charging station management suitable for future smart city applications

CHAPTER 3 : METHODOLOGY

The project begins with RFID-based user authentication using the RC522 RFID module connected to the ESP32 microcontroller. When a valid RFID card is scanned, the ESP32 checks the availability of charging slots using data received from IR sensors. The nearest available slot is automatically assigned to the user, and the servo motor opens the entry gate. Once the vehicle occupies the assigned slot, the corresponding IR sensor detects the vehicle presence and sends the information to the ESP32. The relay module is then activated to simulate EV charging. During charging, parameters such as battery percentage, charging temperature, and estimated charging time remaining are generated and continuously updated. The ESP32 sends all real-time data to Firebase Realtime Database through WiFi communication. The operator dashboard displays complete information about all charging slots, while the user dashboard displays information related only to the assigned slot. All updates are synchronized in real time using Firebase cloud integration.

3.1 Overall System Approach

The system follows a layered approach with three distinct layers working together:

- Hardware Layer: Physical components — ESP32, RFID, IR sensors, relay, servo motor.
- Cloud Layer: Firebase Realtime Database — stores and syncs all live data.
- Web Layer: Operator and User dashboards — display live data fetched from Firebase.

The hardware layer continuously monitors physical conditions and sends data upward to the cloud layer. The web layer reads from the cloud and updates the dashboards in real time. All three layers are always active and in sync.

3.2 Block Diagram

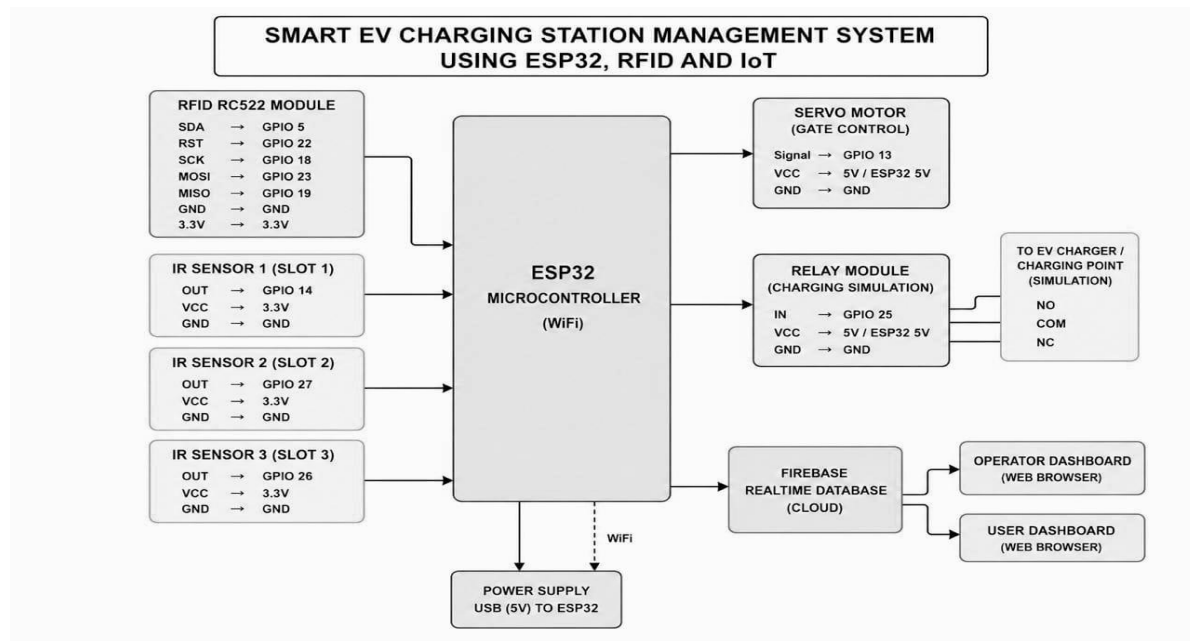


Figure 1: Block diagram (1) Source: Self-created block diagram using project architecture designed by the authors

Description: The ESP32 is at the center. On the input side, it is connected to the RFID RC522 module and three IR sensors. On the output side, it controls the servo motor and relay module. The ESP32 connects to Firebase Realtime Database over WiFi. Firebase serves data to both the Operator Dashboard and the User Dashboard, which are accessed through a web browser.

3.3 Data Flow

The data flows in the following direction

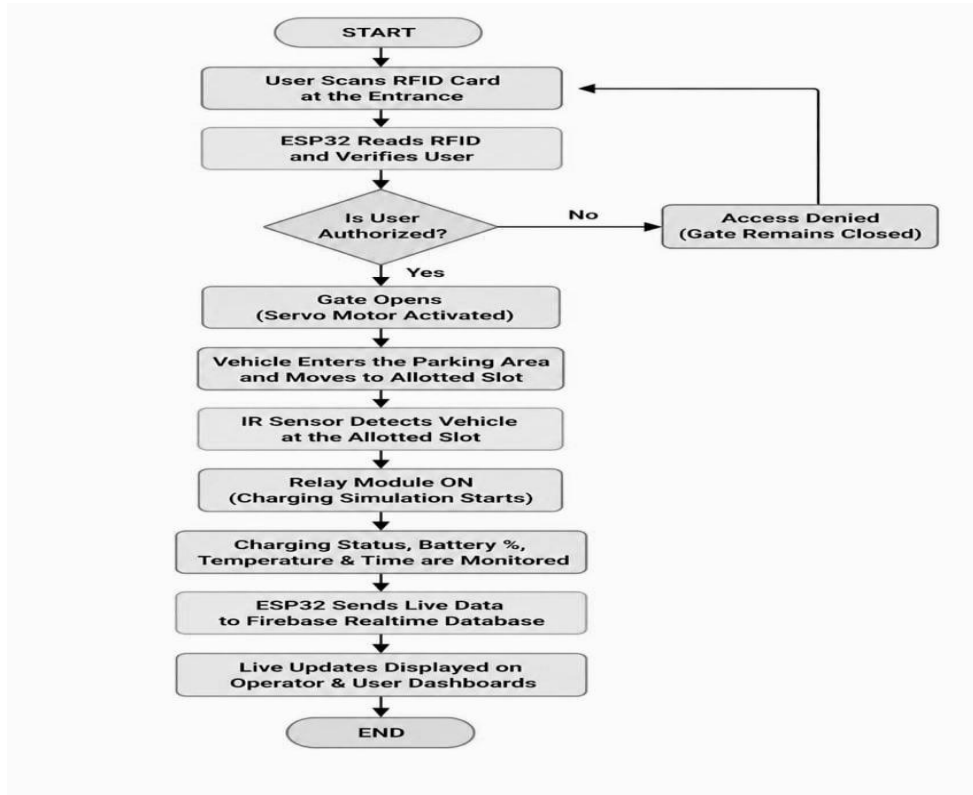


Figure 2 : Data flow (2) Source: Self-created data flow diagram developed for the proposed system methodology

3.4 System Working — Step by Step

3.4.1 RFID Authentication

When a user arrives at the charging station, they hold their RFID card near the RC522 reader. The reader detects the card and reads its unique card ID, called the UID. This UID is sent to the ESP32 via SPI communication. The ESP32 compares this UID with a list of registered UIDs stored in the program. If the UID matches, the user is authenticated and the system proceeds. If the UID does not match, the system denies access and waits for the next card scan. This step is the security gate of the entire system — only registered users can proceed further.

3.4.2 Slot Allocation Logic

After successful authentication, the ESP32 checks the current status of all three charging slots and assigns the nearest available one to the user.

The logic follows this priority order:

IF Slot 1 is Empty → Assign Slot 1

ELSE IF Slot 2 is Empty → Assign Slot 2

ELSE IF Slot 3 is Empty → Assign Slot 3

ELSE → "Parking Full" — deny entry

The assigned slot number is immediately written to Firebase under the key assigned_slot. The User Dashboard picks this up and displays "Go to Slot X" to the user.

If all slots are full, the system writes "Full" to Firebase and the User Dashboard displays "Parking Full. Please try again later."

3.4.3 Gate Automation

Once the slot is assigned, the entry gate opens automatically. The ESP32 sends a PWM signal to the servo motor, rotating it from 0° to 90° — representing the gate opening. The gate stays open for approximately 5 seconds to allow the vehicle to enter. After that, the servo automatically rotates back to 0°, closing the gate. This automation eliminates the need for a physical gate operator and ensures the gate only opens after verified authentication.

3.4.4 Slot Occupancy Detection

All three IR sensors continuously send their output to the ESP32. Each sensor monitors one slot:

When a vehicle is present → IR beam is blocked → Sensor output = LOW → Slot status = "Occupied"

When the slot is empty → IR beam is clear → Sensor output = HIGH → Slot status = **"Empty"**

The ESP32 reads all three sensors in every loop cycle and updates Firebase with the current status of each slot. This ensures the dashboards always reflect the real physical condition of the station.

3.4.5 Charging Simulation

Charging begins only when both of the following conditions are true:

1. RFID authentication was successful.
2. The vehicle is detected in the assigned slot by the IR sensor.

When both conditions are met:

- The relay turns ON, simulating the charging connection.
- charging_status is set to "Charging" in Firebase.
- Battery percentage starts from approximately 20% and increases gradually with each loop cycle.
- Temperature is dynamically generated within a realistic range of 25°C to 45°C.

- Time remaining is calculated based on current battery level and decreases as battery increases.
- When battery reaches 100%, charging stops automatically — relay turns OFF and charging_status becomes "Charging Complete."

If the vehicle is removed from the slot before charging is complete, the IR sensor detects the empty slot, the relay turns OFF, and charging pauses immediately.

3.4.6 Firebase Data Upload

In every loop cycle, the ESP32 uploads the following data to Firebase Realtime Database:

Firestore Key	Description	Example Value
Slot1_status	Status of Slot 1	"Occupied"
slot2_status	Status of Slot 2	"Empty"
slot3_status	Status of Slot 3	"Empty"
Available slots	Number of free slots	2
Occupied slots	Number of occupied slots	1
Assigned slots	Slot assigned to user	1
Charging status	Charging / Not Charging	"Charging"
Battery percentage	Current battery level	67
Temperature	Charging temperature (°C)	38
Time remaining	Estimated minutes left	14

Table 1 Firestore data

The ESP32 uses the Firestore-ESP32 library to write each value to its path in the Firestore JSON tree. All data syncs to the dashboards within 1–2 seconds

CHAPTER 4 : SIMULATION

Before implementing the hardware prototype, the proposed Smart EV Charging Station Management System was tested and verified using the Wokwi online simulation platform. Wokwi is an IoT simulation environment that allows virtual testing of microcontroller-based projects such as ESP32 and Arduino systems. The simulation helped in verifying the logic, sensor behavior, and communication process before connecting the actual hardware components.

In the simulation, the ESP32 microcontroller was configured along with virtual components such as RFID module, IR sensors, servo motor, and relay module. The working logic of RFID authentication, automatic slot allocation, gate control, and charging simulation was tested successfully. The simulation also verified the interaction between the ESP32 and Firebase Realtime Database for real-time data updates.

The IR sensors were simulated to detect slot occupancy conditions, while the relay module simulated the charging process. The servo motor operation for automatic gate opening and closing was also verified. Battery percentage, charging temperature, and charging time remaining values were generated dynamically during the simulation to observe charging behavior.

The Wokwi simulation provided several advantages during project development. It reduced hardware testing errors, helped in debugging the ESP32 code, and allowed verification of the complete system logic before final hardware implementation. The simulation also improved development efficiency by enabling quick modifications and testing of the program logic in a virtual environment.

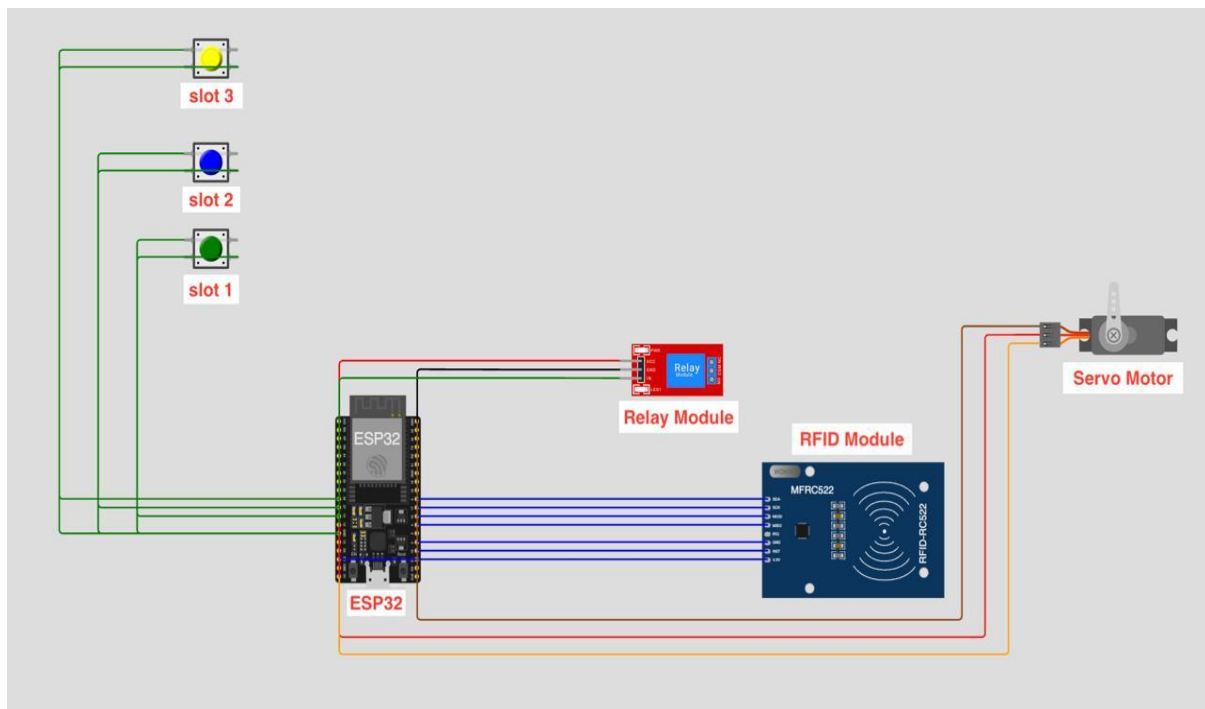


Figure 3 : Simulation (3) Source: Screenshot captured from Wokwi ESP32 Simulation Environment

CHAPTER 5 : HARDWARE MODEL

5.1 Components Used

1. ESP32 Microcontroller

The ESP32 is the brain of the entire system. It is a 32-bit dual-core microcontroller with built-in WiFi and Bluetooth. It handles all sensor reading, actuator control, and Firebase communication. It operates at 3.3V logic and runs at up to 240 MHz. In this project it reads RFID data via SPI, reads IR sensor states via digital GPIO pins, controls the servo motor via PWM, controls the relay via a digital output pin, and sends all data to Firebase over WiFi.

2. RFID RC522 Module

The RC522 is a contactless RFID reader that operates at 13.56 MHz. It communicates with the ESP32 via SPI protocol. When a user holds an RFID card near the module, it reads the card's unique UID and sends it to the ESP32 for verification. Each registered user has a unique RFID card with a pre-stored UID in the system

3. IR Sensors (×3)

Three infrared sensors are used — one for each charging slot. They work by emitting an IR beam between a transmitter and receiver. When a vehicle (or object) is present in the slot, the beam is interrupted and the sensor output changes, which the ESP32 detects as an occupancy signal. These sensors are reliable, inexpensive, and easy to connect to ESP32 GPIO pins

4. Servo Motor (SG90)

The SG90 servo motor is used as the automatic entry gate. It is controlled by PWM signals from the ESP32. It rotates to 90° when the gate opens and returns to 0° when the gate closes. It operates on 5V and is lightweight and compact, making it ideal for prototype-level gate simulation.

5. Relay Module (5V Single Channel)

The relay module simulates the EV charging connection. It is an electrically operated switch controlled by a 3.3V GPIO signal from the ESP32. When the relay is ON, it represents active charging. When OFF, it represents charging stopped or not started.

Component	Model	Quantity	Role
Microcontroller	ESP32	1	Central controller
RFID Module	RC522	1	User authentication
IR Sensor	Digital IR Module	3	Slot occupancy detection
Servo Motor	SG90	1	Automatic gate control
Relay Module	5V Single Channel	1	Charging simulation
Breadboard	Standard	1	Circuit assembly
Jumper Wires	M-M / M-F	Multiple	Connections

Table 2 Components

5.2 Hardware Setup

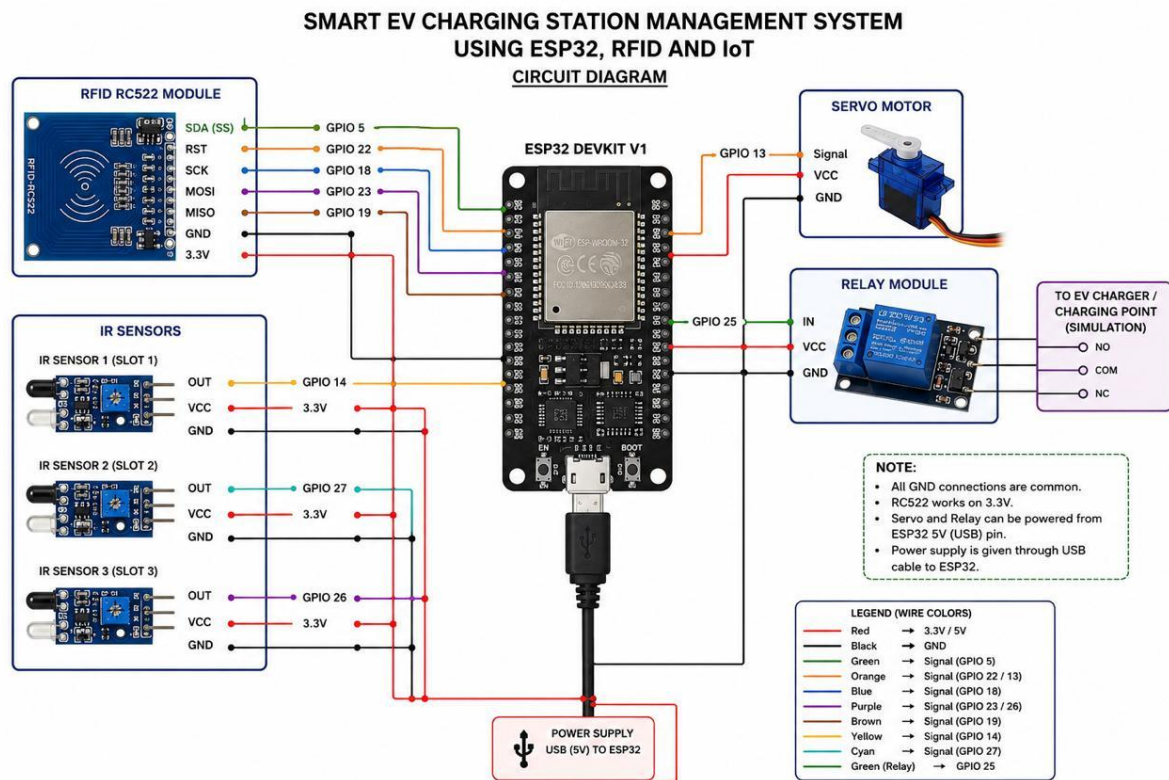


Figure 4 : Hardware setup

CHAPTER 6 : SOFTWARE AND CLOUD INTEGRATION

6.1 Arduino IDE and Libraries

Arduino IDE is used to write and upload the complete program to the ESP32. The code handles all hardware logic, charging simulation, and Firebase communication.

Key libraries used:

Library	Purpose
WiFi.h	Connects ESP32 to WiFi network
MFRC522.h	Reads RFID card UID via SPI
ESP32Servo.h	Controls servo motor via PWM
Firebase ESP Client.h	Sends and receives data from Firebase

Table 3 : Libraries used

Code logic overview:

The setup() function initializes WiFi, Firebase, RFID module, servo motor, relay pin, and IR sensor pins. The loop() function runs continuously and performs the following tasks in order:

- Check for RFID card → verify UID → assign slot → open gate.
- Read all three IR sensors → update slot status in Firebase.
- Check if vehicle is in assigned slot → activate relay → start charging simulation.
- Update battery %, temperature, time remaining → upload to Firebase.
- If battery = 100% → stop charging → deactivate relay

6.2 Firebase Realtime Database

Firebase Realtime Database is a cloud-hosted, NoSQL database service provided by Google as part of the Firebase platform. Unlike traditional databases that require a dedicated server and complex query language, Firebase stores all data as a single JSON (JavaScript Object Notation) tree and allows any connected client — whether a microcontroller, web page, or mobile application — to read or write data in real time with minimal latency.

In this project, Firebase Realtime Database acts as the central communication bridge between the hardware layer (ESP32) and the web layer (dashboards). The ESP32 writes live operational data to Firebase, and both dashboards retrieve this data instantly to display it to the operator and the user.

Why Firebase Was Chosen:

Firebase was selected for this project for the following reasons:

It provides a free tier that is more than sufficient for a prototype-level project. It supports real-time data synchronization, meaning any change written by the ESP32 is immediately reflected on the dashboards without requiring a page refresh or polling mechanism. The Firebase-ESP32 library provides simple, ready-to-use functions to write and read data from the database without complex HTTP handling. Firebase's REST API allows JavaScript-based dashboards to fetch live data with just a few lines of code. The Firebase console provides a visual interface to monitor the database structure and values, which was extremely useful during testing and debugging

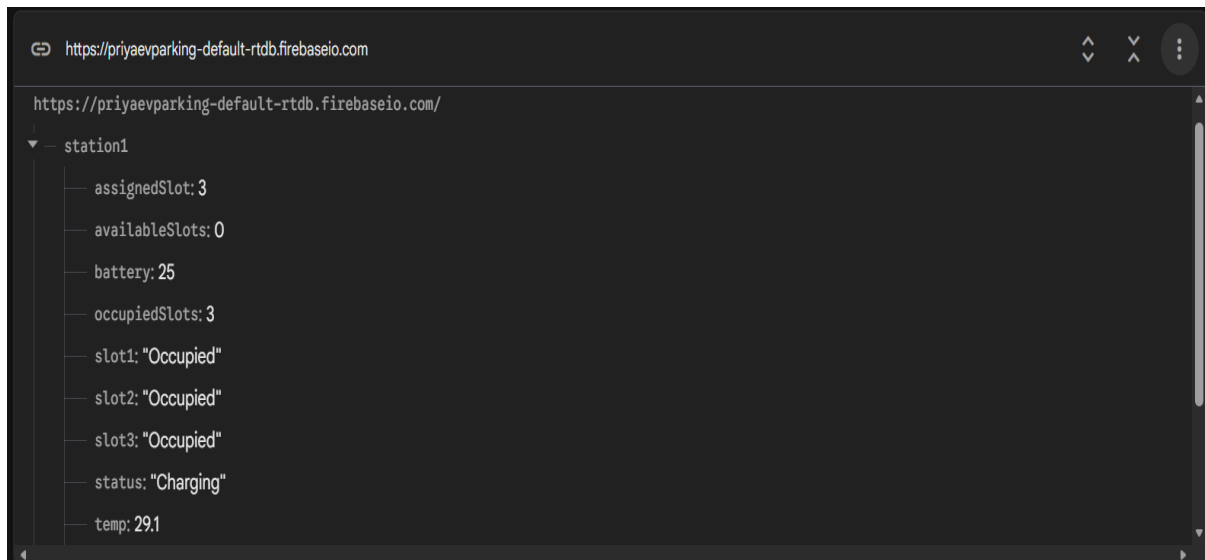


Figure 5 : Firebase interface (4) Screenshot captured from Firebase Realtime Database Console.

6.3 Dashboard Design

Two separate dashboards are developed — one for the operator and one for the user. Both are built using HTML, CSS, and JavaScript and fetch live data from Firebase.

6.3.1 Operator Dashboard

The Operator Dashboard is designed specifically for the use of the station manager or operator, who requires complete and continuous visibility into the operational state of the entire charging station. This dashboard can be accessed remotely from any device with a web browser and an internet connection, enabling the operator to monitor the station without being physically present

Features:

The operator dashboard is designed to provide complete real-time monitoring of the EV charging station. It displays the status of all charging slots using color-coded cards, where green indicates an empty slot and red indicates an occupied slot. Charging-related parameters such as battery percentage, charging status, charging temperature, and estimated charging time remaining are continuously updated. A visual progress bar is used to represent the battery charging level for better understanding. All information is fetched from Firebase Realtime Database and updated automatically in real time without requiring page refresh.

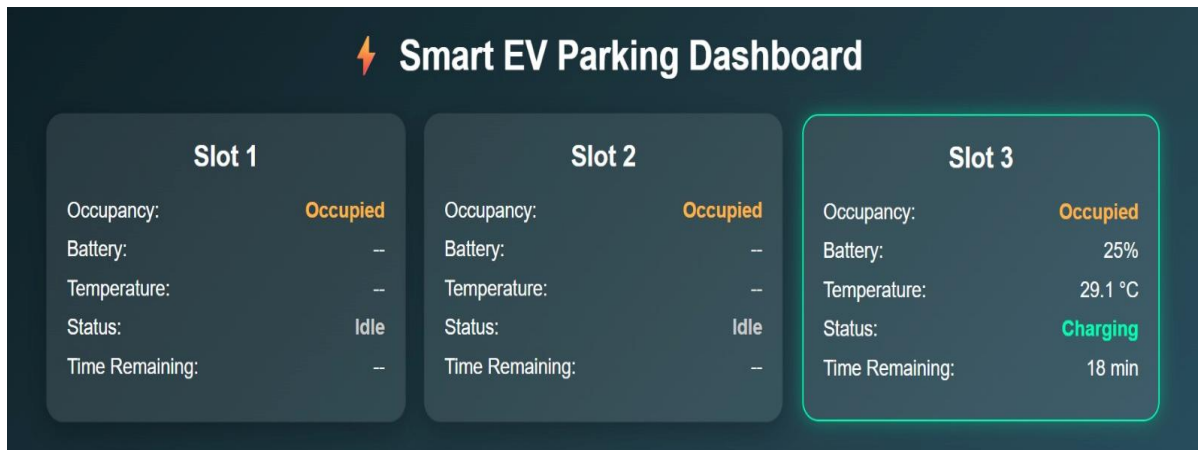


Figure 6 : Operator dashboard (5) Source: Self-developed Operator Dashboard using HTML, CSS, JavaScript, and Firebase integration

6.3.2 User Dashboard

The User Dashboard is for the EV owner. It shows only the information relevant to their session.

Features:

The user dashboard is developed to provide charging information specifically for the authenticated user. After successful RFID authentication, the dashboard displays the assigned charging slot with a message such as “Go to Slot X.” If all slots are occupied, a “Parking Full” message is displayed. Once the vehicle occupies the assigned slot, the dashboard shows charging details including charging status, battery percentage with a progress indicator, charging temperature, and estimated charging time remaining. All data is fetched from Firebase Realtime Database and updates automatically in real time.



Figure 7 : User dashboard (6) Source: Self-developed User Dashboard using HTML, CSS, JavaScript, and Firebase integration

CHAPTER 7 : RESULTS

7.1 Test Results

The complete system was tested across all its functional modules. Results are presented below.

Test 1 — RFID Authentication:

Test Case	Card Type	Result
Registered RFID card	Valid UID	Access Granted, Gate Opens
Unregistered RFID card	Unknown UID	Access Denied
No card present	No input	System waits, no action

Table 4 : RFID authentication test

All three cases behaved correctly. Response time from card scan to gate opening was less than 1 second.

Test 2 — Slot Allocation:

Slot 1	Slot 2	Slot 3	Result
Empty	Empty	Empty	Slot 1 Assigned
Occupied	Empty	Empty	Slot 2 Assigned

Table 5 : Slot allocation test

Priority-based allocation worked correctly in all scenarios.

Test 3 — IR Sensor Occupancy Detection:

All three IR sensors correctly detected vehicle presence and absence. Slot status updated in Firebase within 1–2 seconds of physical change.

Test 4 — Charging Simulation:

Parameter	Observed Behavior
Starting battery %	~20–25%
Battery increase	Gradual, per loop cycle
Temperature range	25°C to 45°C (dynamic)
Time remaining	Decreased as battery increased
Auto stop at 100%	Worked correctly
Relay on vehicle removal	Turned OFF immediately

Table 6 : Charging simulation test

Test 5 — Firebase Synchronization:

All data fields updated correctly in Firebase. Average sync delay observed was under 2 seconds, which is acceptable for this application.

Test 6 — Operator Dashboard:

All slot statuses, battery %, charging status, temperature, and time remaining displayed correctly. Dashboard refreshed automatically with live data.

Test 7 — User Dashboard:

Assigned slot, charging status, and all parameters displayed correctly. Parking Full message was correctly shown when all slots were occupied.

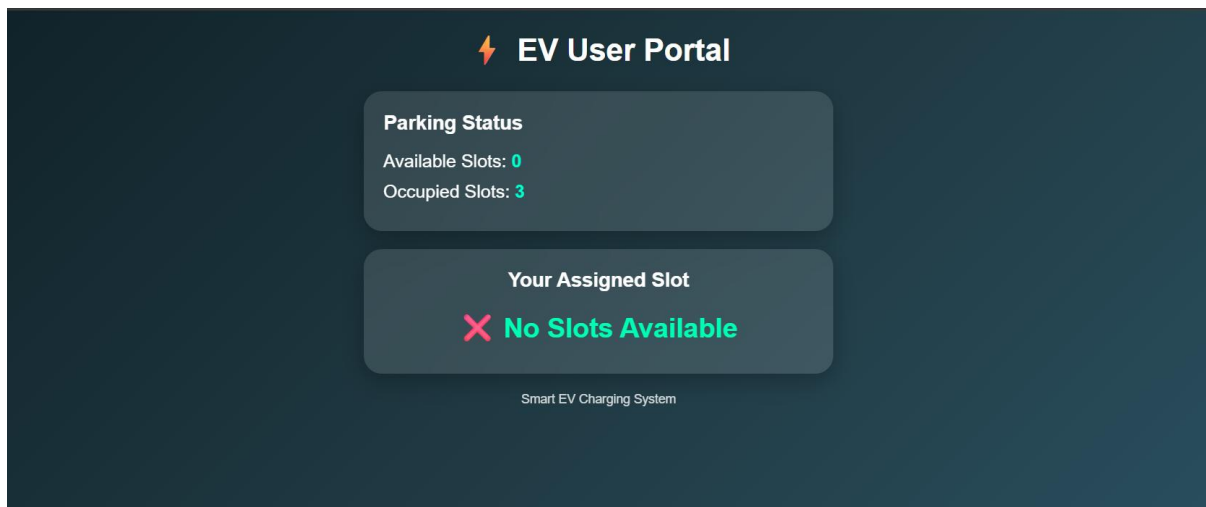


Figure 8 : EV user portal while slots unavailability (8) Self developed user dashboard screenshot

7.2 Observations

The RFID authentication system performed accurately during testing, with no false acceptances or rejections observed. The servo motor successfully controlled the gate operation by opening and closing smoothly after valid authentication. The IR sensors provided reliable slot occupancy detection under normal laboratory conditions without producing false readings. Firebase Realtime Database maintained stable and continuous synchronization between the hardware and web dashboards. Both the operator and user dashboards displayed real-time updates without noticeable delay. The charging simulation, including battery percentage, charging temperature, and estimated time remaining, behaved realistically throughout the testing process. The system also handled special cases effectively, such as immediately pausing charging when a vehicle was removed from the assigned slot during charging.

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project successfully designed and implemented a Smart EV Charging Station Management System using ESP32, RFID, IR sensors, relay module, servo motor, Firebase Realtime Database, and web dashboards. All the objectives of the project were achieved:

- RFID-based authentication was implemented and tested successfully.
- Automatic slot allocation with priority logic worked correctly for all slot conditions.
- The servo motor gate automated the entry process reliably.
- IR sensors accurately detected slot occupancy in real time.
- Charging simulation with dynamic battery, temperature, and time data worked as expected.
- All live data was successfully uploaded to Firebase and made available to both dashboards.
- The Operator Dashboard provided a complete and accurate view of the station.
- The User Dashboard showed personalized and relevant information to the authenticated user.

This project demonstrates that a fully functional, automated, and cloud-connected EV charging station management system can be built using affordable IoT components. It shows the practical potential of combining embedded systems, cloud databases, and web technologies to solve real-world infrastructure problems.

As India and the world continue to expand their EV ecosystems, systems like this can play a meaningful role in making EV charging smarter, more efficient, and more user-friendly.

8.2 Future Scope

This project can be extended and improved in the following ways:

- 1. Mobile Application:** A smartphone app connected to Firebase can provide the same operator and user dashboard features on Android or iOS, making monitoring more convenient.
- 2. Real Charging Hardware:** Instead of simulation, the system can be connected to actual EV chargers with current sensors and energy meters to measure real power consumption.
- 3. Payment Integration:** An online payment module can be added so users are charged based on the amount of electricity consumed or the time they charge.

4. Multiple User Support: The system can be extended to support multiple simultaneous users with individual RFID cards, Firebase user accounts, and session management.

5. SMS / Email Notifications: Automated alerts can be sent to users when charging is complete, or to operators when a slot is occupied for too long.

6. Solar Power Integration: The system can be upgraded to monitor solar panel output and battery storage alongside EV charging, making the station energy-independent.

7. Expanded Slot Capacity: By adding more IR sensors and updating Firebase and dashboard logic, the system can be scaled to handle more than three slots easily.

REFERENCES

1. Espressif Systems. (2023). *ESP32 Technical Reference Manual*. Espressif Systems. <https://www.espressif.com/>
2. Google Firebase. (2024). *Firebase Realtime Database Documentation*. Google LLC. <https://firebase.google.com/docs/database>
3. NXP Semiconductors. (2016). *MFRC522 Standard Performance MIFARE and NTAG Frontend Datasheet*. NXP Semiconductors. <https://www.nxp.com/>
4. Ahuja, R., & Dua, A. (2022). Smart EV charging system using IoT and cloud platforms. *International Journal of Electrical Engineering and Technology*, 13(2), 45–53.
5. Singh, P., & Kaur, G. (2021). IoT-based smart parking system using IR sensors and ESP8266. *Journal of Embedded Systems and IoT*, 5(1), 12–19.
6. Kumar, A., & Mehta, R. (2023). RFID-based authentication for smart city applications. *IEEE Transactions on Intelligent Transportation Systems*, 24(5), 3120–3130.
7. Arduino. (2024). *Arduino IDE Reference Documentation*. Arduino AG. <https://www.arduino.cc/>
8. Mozilla Developer Network. (2024). *HTML, CSS, and JavaScript Reference*. MDN Web Docs. <https://developer.mozilla.org/>
9. Ministry of Heavy Industries, Government of India. (2022). *FAME India Scheme – Phase II Progress Report*. Department for Promotion of Industry and Internal Trade, New Delhi.
10. Srinivasan, K., & Nair, V. (2022). Real-time monitoring of EV charging stations using cloud IoT platforms. *International Conference on Emerging Technologies in Smart Grids (ICETSG)*, Bengaluru, India, pp. 118–124

